

HospFinder DSS

Rekomendasi & Navigasi Evakuasi Rumah Sakit Kawasan Denpasar

Anggota Kelompok:

1. Nama : I Putu Agus Adi Wiranata | NIM : [2501010068] | Akun GitHub : [@Wiranz]
2. Nama : I Putu Vivekananda Gosvami | NIM : [2501010350] | Akun GitHub : [@Vivekananda2501]
3. Nama : I Gede Arya Desta Adi Wiguna | NIM : [2501010083] | Akun GitHub : [@Adi4259j]

Studi Kasus

- 01 Latar Belakang: Dalam kondisi darurat, pasien atau paramedis sering kesulitan menentukan Rumah Sakit (RS) rujukan yang optimal di kawasan Denpasar
- 02 Kompleksitas Keputusan: Pemilihan RS yang tepat tidak hanya bergantung pada jarak fisik terdekat, melainkan juga harus mempertimbangkan kualitas pelayanan (rating) dan tipe fasilitas medis.
- 03 Solusi Sistem: Membangun Decision Support System (DSS) berbasis Graph Database untuk mencari rute terpendek dan mengklasifikasikan (Tiering) Rumah Sakit sebagai rekomendasi yang cerdas.

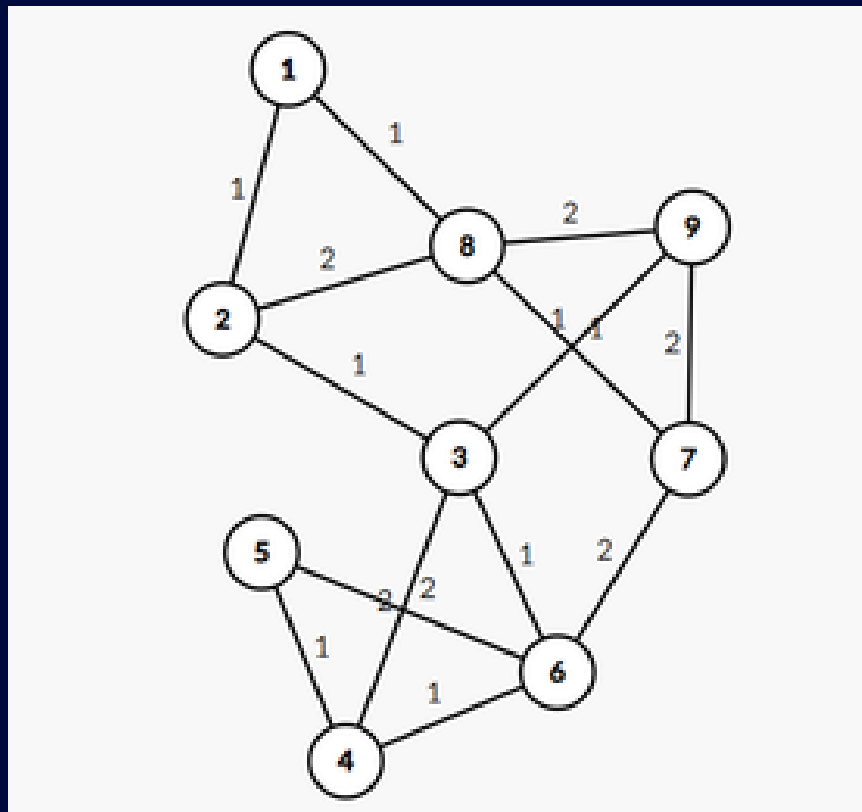


Model Graph

01

Jenis Graph:

Sistem ini memodelkan peta menggunakan **Weighted Undirected Graph** (Graf berbobot dan tak berarah).



02

Representasi Data:

- **Node (Simpul):** Merepresentasikan Titik Wilayah Dasar (seperti Tabanan, Mengwi, Canggu, Denpasar) dan Titik Rumah Sakit tujuan.
- **Edge (Garis):** Merepresentasikan topologi rute yang menghubungkan antar-node.
- **Weight (Bobot):** Nilai jarak fisik dalam satuan kilometer (km) yang dihitung menggunakan formula matematika Haversine berdasarkan koordinat lintang dan bujur (latitude/longitude).

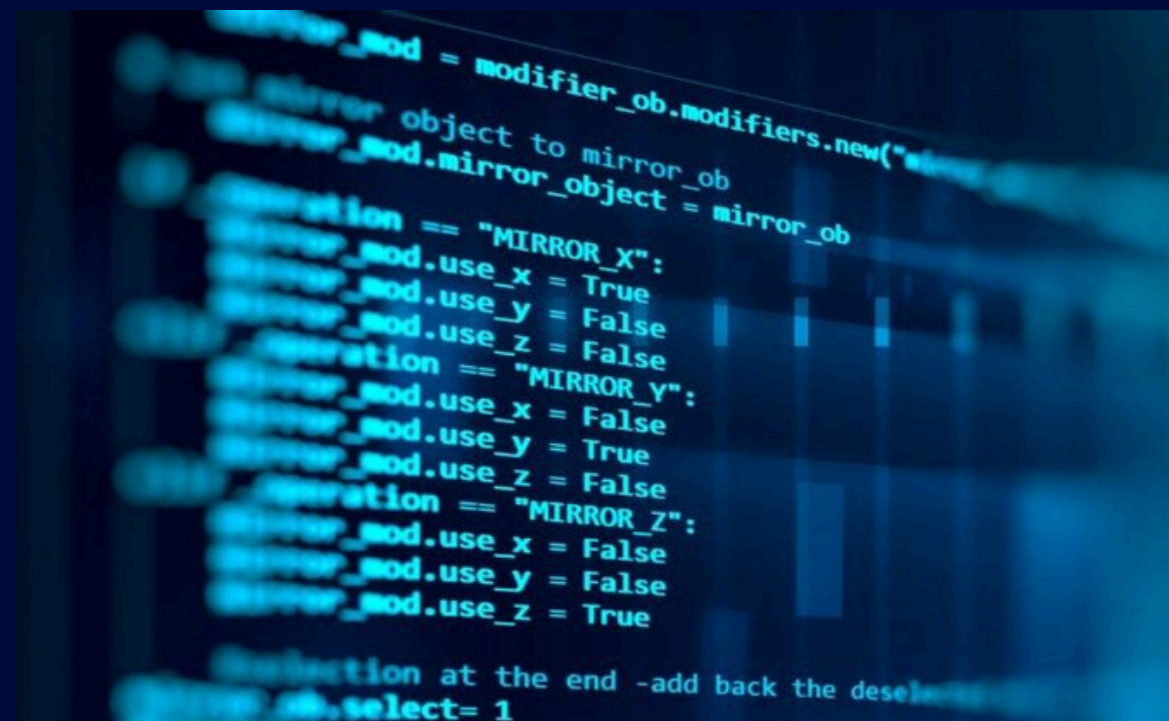
Algoritma Utama

Sistem kami memadukan Algoritma Struktur Data dengan Machine Learning dan AI:

Algoritma Dijkstra:

Digunakan untuk pencarian rute terpendek

(shortest path). Diimplementasikan menggunakan struktur Priority Queue (Min-Heap) agar komputasi pencarian berjalan sangat cepat.



```
mirror_mod = modifier_ob.modifiers.new("mirror_mod")
# Add mirror object to mirror_ob
mirror_mod.mirror_object = mirror_ob

# Operation == "MIRROR_X":
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
# Operation == "MIRROR_Y":
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
# Operation == "MIRROR_Z":
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

# Selection at the end -add back the deselected
group_ob.select= 1
```

K-Means Clustering:

Algoritma Machine Learning yang membagi data RS menjadi beberapa kelompok (Tier) berdasarkan metrik jarak, rating, dan biaya.

Generative AI (Groq LLaMA 3.1):

Fitur bonus berupa sistem pakar yang memberikan ulasan logistik medis secara naratif berdasarkan data RS dan jarak evakuasi yang dipilih.

Demo Sistem

Langkah 1 (Input):

Pengguna memilih lokasi awal keberangkatan pasien (misal: Tabanan) Dan Juga Tipe Rumah Sakit (misal : Tipe B).

Langkah 2 (Processing):

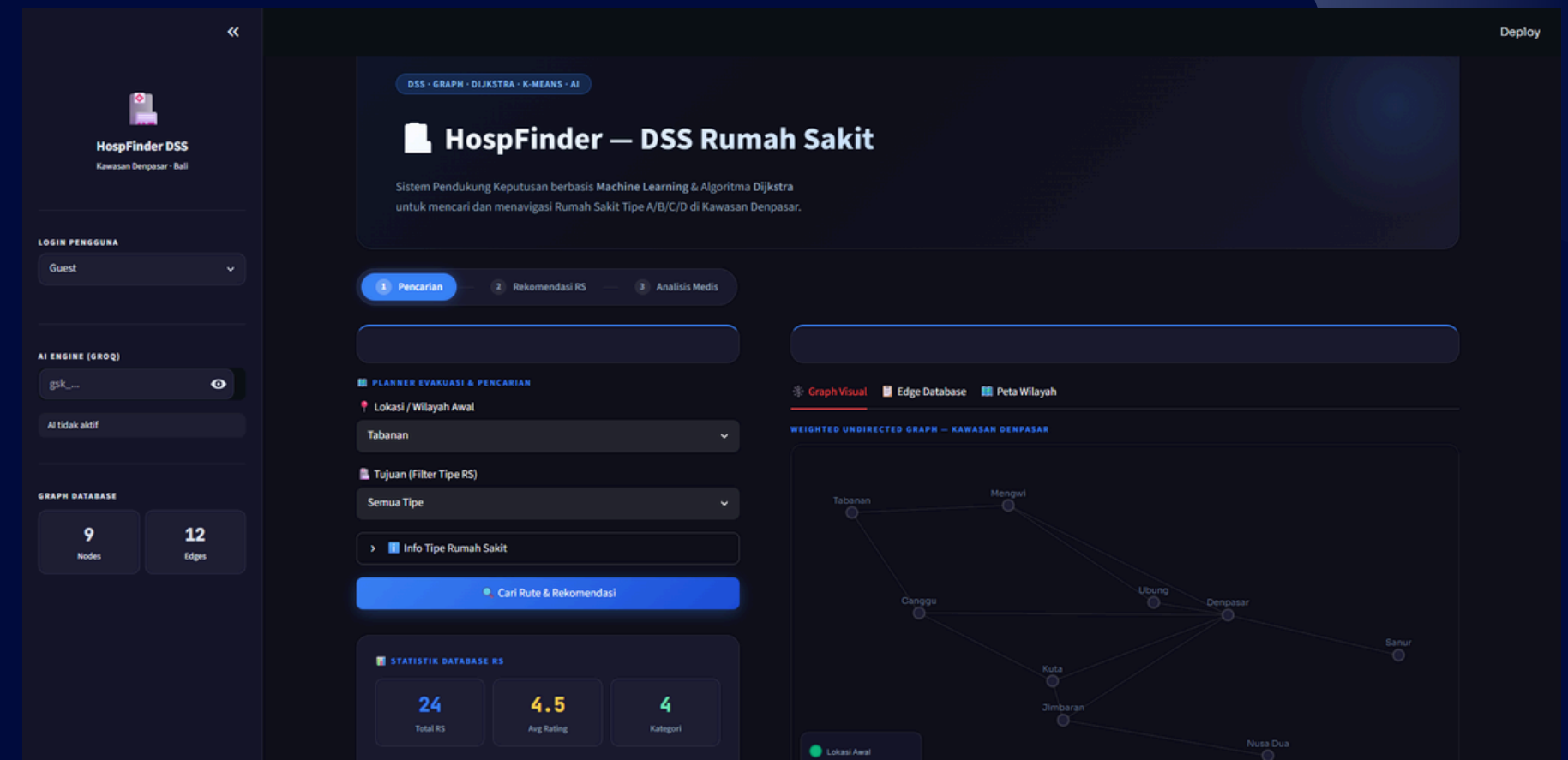
Sistem secara otomatis menghitung Dijkstra ke seluruh RS dan memproses pemeringkatannya dengan K-Means.

Langkah 3 (Rekomendasi):

Menampilkan metrik "Tier List" (Tier S untuk prioritas utama) dan mengarahkan pengguna memilih RS tujuan.

Langkah 4 (Visualisasi) :

Menampilkan Dashboard berisi detail RS, Graph interaktif rute tercepat (menggunakan Matplotlib), dan narasi kecerdasan buatan (AI).



Sistem kami memadukan Algoritma Struktur Data dengan Machine Learning dan AI

Hasil Analisis

01 **kecepatan & Performa:**

Struktur Graph dengan implementasi Min-Heap terbukti mampu memproses navigasi ke 24 Rumah Sakit sekaligus dalam hitungan milidetik.

02 **Akurasi Keputusan:**

DSS bekerja optimal karena tidak sekadar menyarankan RS terdekat secara buta, melainkan memberikan peringkat (Tier) berbasis Machine Learning yang objektif.

03 **Transparansi:**

Sistem menyajikan "Log Komputasi Dijkstra" (langkah demi langkah algoritma) secara visual kepada pengguna.



Kendala dan Solusi

Kendala:

Bagaimana menghubungkan koordinat geografis (lokasi asli RS) ke dalam Base Graph wilayah yang masih bersifat statis.

Kesimpulan:

Implementasi Graph sangat krusial sebagai fondasi utama dalam sistem navigasi dan pengambilan keputusan darurat di dunia nyata.

Solusi:

Kami menerapkan fitur penambahan Node & Edge sementara secara dinamis saat sistem berjalan (runtime). Sistem akan mencari base node terdekat dari RS menggunakan Haversine, membuat garis penghubung (edge) baru dengan bobot jarak aslinya, lalu menjalankan komputasi Dijkstra dari lokasi awal ke RS tersebut.

Apakah Ada Pertanyaan?

TERIMA KASIH